

Numerical Methods for Root-Finding and Nonlinear Eigenvalue Problems Report

DA7067 Computational Mathematics

Onyinyechi Sonia Bäckström

Department of Mathematics
Stockholm University

October 2026

Task 1: Contour-integration Methods

Find all the roots of

$$f(z) = e^{2z} \cos(5z) - 5z^2$$

inside the unit disk

$$K = \{z \in \mathbb{C} : |z| < 1\}.$$

(a) How many roots does f have in K ?

We find a root, a value z for which $f(z) = 0$.

In the given function, f has no zeros or poles on the boundary.

Using Cauchy's Argument Principle, if f has no zeros or poles on the boundary,

$$N = \frac{1}{2\pi i} \oint_{\partial K} \frac{f'(z)}{f(z)} dz,$$

where N is the number of roots inside the contour.

We first find $f'(z)$.

Using the product rule,

$$(uv)' = u'v + uv',$$

we have

$$\frac{d}{dz} (e^{2z} \cos(5z)) = 2e^{2z} \cos(5z) - 5e^{2z} \sin(5z).$$

Also,

$$\frac{d}{dz} (5z^2) = 10z.$$

Therefore,

$$f'(z) = e^{2z} (2 \cos(5z) - 5 \sin(5z)) - 10z.$$

The root-counting integral is

$$N = \frac{1}{2\pi i} \oint_{|z|=1} \frac{e^{2z} (2 \cos(5z) - 5 \sin(5z)) - 10z}{e^{2z} \cos(5z) - 5z^2} dz.$$

To evaluate the contour integral numerically, we parameterize the unit circle,

$$|z| = 1,$$

by using an angle θ :

$$z = e^{i\theta}, \quad 0 \leq \theta \leq 2\pi.$$

Differentiating

$$z = e^{i\theta}$$

with respect to θ gives

$$\frac{dz}{d\theta} = ie^{i\theta}.$$

Therefore,

$$dz = ie^{i\theta} d\theta.$$

This describes one complete trip around the unit circle.

Substituting both z and dz in

$$N = \frac{1}{2\pi i} \oint_{|z|=1} \frac{f'(z)}{f(z)} dz,$$

we obtain

$$N = \frac{1}{2\pi i} \int_0^{2\pi} \frac{f'(e^{i\theta})}{f(e^{i\theta})} (ie^{i\theta}) d\theta.$$

Hence,

$$N = \frac{1}{2\pi} \int_0^{2\pi} e^{i\theta} \frac{f'(e^{i\theta})}{f(e^{i\theta})} d\theta.$$

For convenience, let

$$h(\theta) = e^{i\theta} \frac{f'(e^{i\theta})}{f(e^{i\theta})}.$$

Then

$$N = \frac{1}{2\pi} \int_0^{2\pi} h(\theta) d\theta.$$

The integral above cannot be evaluated exactly by integrating around the contour. Instead, we approximate it numerically. The trapezoidal rule approximates the area under a curve using trapezoids.

For equally spaced points, the trapezoidal rule approximates

$$\int_a^b h(\theta) d\theta$$

by a weighted sum of function values.

In the given function, the interval is

$$0 \leq \theta \leq 2\pi,$$

and the parametrization of the circle is periodic because

$$e^{i0} = e^{i2\pi} = 1.$$

Therefore, we use the periodic trapezoidal rule.

Suppose we use M equally spaced points. To go around the circle once, θ goes from 0 to 2π . Thus, the total interval has length 2π .

Therefore, the spacing between consecutive values of θ is

$$\Delta\theta = \frac{2\pi}{M}.$$

Starting from $\theta_0 = 0$, the j -th point is

$$\theta_j = j\Delta\theta = j\frac{2\pi}{M} = \frac{2\pi j}{M},$$

where

$$j = 0, 1, 2, \dots, M-1.$$

Since

$$z = e^{i\theta},$$

we have

$$z_j = e^{i\theta_j} = e^{i(2\pi j/M)}.$$

Therefore,

$$z_j = e^{2\pi i j/M}.$$

Here, z_j represents M equally spaced points on the unit circle.

Now we apply the periodic trapezoidal rule. For equally spaced points, the trapezoidal approximation of the integral is

$$\int_0^{2\pi} h(\theta) d\theta \approx \Delta\theta \sum_{j=0}^{M-1} h(\theta_j).$$

Using

$$\Delta\theta = \frac{2\pi}{M},$$

we obtain

$$\int_0^{2\pi} h(\theta) d\theta \approx \frac{2\pi}{M} \sum_{j=0}^{M-1} h(\theta_j).$$

Recall that

$$h(\theta) = e^{i\theta} \frac{f'(e^{i\theta})}{f(e^{i\theta})}.$$

Thus,

$$N \approx \frac{1}{2\pi} \frac{2\pi}{M} \sum_{j=0}^{M-1} e^{i\theta_j} \frac{f'(e^{i\theta_j})}{f(e^{i\theta_j})}.$$

Since

$$z_j = e^{i\theta_j},$$

we obtain

$$N \approx \frac{1}{M} \sum_{j=0}^{M-1} z_j \frac{f'(z_j)}{f(z_j)}.$$

The numerical approximation of the root count is therefore

$$N \approx \frac{1}{M} \sum_{j=0}^{M-1} z_j \frac{f'(z_j)}{f(z_j)}, \quad z_j = e^{2\pi i j / M}.$$

Using the periodic trapezoidal rule with M equispaced points gives

$$N \approx \frac{1}{M} \sum_{j=0}^{M-1} \frac{f'(z_j)}{f(z_j)} z_j, \quad z_j = e^{2\pi i j / M}.$$

The convergence with M is:

M	Approximation of N
64	3.9229395552
128	4.0012936113
256	3.9999928863
512	4.0000000000
1024	4.0000000000
2048	4.0000000000

Table 1: Convergence of the numerical root count as the number of quadrature points is increased.

The result approaches the integer value 4. Therefore, the contour integral indicates that there are four roots inside the unit disk:

$$N = 4.$$

The convergence table also illustrates the assignment's warning that a sufficiently large number of quadrature points is needed.

(b) Finding the roots

From Task 1(a), the function

$$f(z) = e^{2z} \cos(5z) - 5z^2$$

has $N = 4$ roots inside the unit disk

$$K = \{z : |z| < 1\}.$$

We define the contour moments

$$S_p = \frac{1}{2\pi i} \oint_{|z|=1} z^p \frac{f'(z)}{f(z)} dz, \quad p = 0, \dots, 2N - 1.$$

Since $N = 4$, we need

$$p = 0, 1, \dots, 7.$$

Therefore, we calculate

$$S_0, S_1, S_2, S_3, S_4, S_5, S_6, S_7.$$

Let the four roots inside the unit circle be denoted by

$$r_1, r_2, r_3, r_4.$$

he logarithmic derivative $f'(z)/f(z)$ has a simple pole at each simple root r_k , with residue equal to one. Therefore, by the residue theorem,

$$S_p = \sum_{k=1}^4 r_k^p = r_1^p + r_2^p + r_3^p + r_4^p.$$

Numerical evaluation of the moments

As in Task 1(a), the unit circle is parameterized by using

$$z = e^{i\theta}, \quad 0 \leq \theta \leq 2\pi,$$

with

$$dz = ie^{i\theta} d\theta.$$

Substituting z and dz into the moment integral, S_p , we have:

$$S_p = \frac{1}{2\pi i} \int_0^{2\pi} (e^{i\theta})^p \frac{f'(e^{i\theta})}{f(e^{i\theta})} ie^{i\theta} d\theta.$$

$$S_p = \frac{1}{2\pi} \int_0^{2\pi} e^{ip\theta} \frac{f'(e^{i\theta})}{f(e^{i\theta})} e^{i\theta} d\theta,$$

Hence,

$$S_p = \frac{1}{2\pi} \int_0^{2\pi} e^{i(p+1)\theta} \frac{f'(e^{i\theta})}{f(e^{i\theta})} d\theta.$$

Now, we use M equally spaced quadrature points

$$\theta_j = \frac{2\pi j}{M}, \quad j = 0, \dots, M-1,$$

and define

$$z_j = e^{i\theta_j} = e^{2\pi i j / M}.$$

where, z_j denotes the quadrature points on the unit circle.

Applying the periodic trapezoidal rule after choosing M equally spaced points around the circle gives:

$$S_p \approx \frac{1}{2\pi} \frac{2\pi}{M} \sum_{j=0}^{M-1} e^{i(p+1)\theta_j} \frac{f'(e^{i\theta_j})}{f(e^{i\theta_j})}.$$

$$S_p \approx \frac{1}{M} \sum_{j=0}^{M-1} e^{i(p+1)\theta_j} \frac{f'(e^{i\theta_j})}{f(e^{i\theta_j})}.$$

Since

$$z_j = e^{i\theta_j},$$

$$e^{i(p+1)\theta_j} = z_j^{p+1}.$$

Thus,

$$S_p \approx \frac{1}{M} \sum_{j=0}^{M-1} z_j^{p+1} \frac{f'(z_j)}{f(z_j)}.$$

The function and its derivative used in the computation are

$$f(z) = e^{2z} \cos(5z) - 5z^2$$

and

$$f'(z) = e^{2z} (2 \cos(5z) - 5 \sin(5z)) - 10z.$$

For $p = 0, \dots, 7$, the moments are therefore computed numerically using the formula above. We substitute this into the moment formula.

For $p = 0$,

$$S_0 \approx \frac{1}{M} \sum_{j=0}^{M-1} z_j \frac{f'(z_j)}{f(z_j)}.$$

Construction of the Hankel matrices

The moments are assembled into the two matrices

$$H_0 := (S_{i+j})_{i,j=0}^{N-1}, \quad H_1 := (S_{i+j+1})_{i,j=0}^{N-1}.$$

Since $N = 4$,

$$i, j = 0, 1, 2, 3.$$

and we have the moments, we can build H_0 and H_1 :

For $i = 0$, we obtain the first row by varying j :

$$\begin{aligned} S_{0+0} &= S_0, \\ S_{0+1} &= S_1, \\ S_{0+2} &= S_2, \\ S_{0+3} &= S_3. \end{aligned}$$

Therefore, the first row of H_0 is

$$(S_0 \quad S_1 \quad S_2 \quad S_3).$$

Similarly, for $i = 1$,

$$\begin{aligned} S_{1+0} &= S_1, \\ S_{1+1} &= S_2, \\ S_{1+2} &= S_3, \\ S_{1+3} &= S_4. \end{aligned}$$

Doing the same for all rows gives

$$H_0 = \begin{pmatrix} S_0 & S_1 & S_2 & S_3 \\ S_1 & S_2 & S_3 & S_4 \\ S_2 & S_3 & S_4 & S_5 \\ S_3 & S_4 & S_5 & S_6 \end{pmatrix}.$$

Again,

$$i, j = 0, 1, 2, 3,$$

Similarly, the second matrix is obtained

$$H_1 = \begin{pmatrix} S_1 & S_2 & S_3 & S_4 \\ S_2 & S_3 & S_4 & S_5 \\ S_3 & S_4 & S_5 & S_6 \\ S_4 & S_5 & S_6 & S_7 \end{pmatrix}.$$

Now we have two 4×4 matrices. The roots are then obtained from the generalized eigenvalue problem

$$H_1 v = \lambda H_0 v.$$

Approximate roots

Using $M = 1024$ quadrature points, the generalized eigenvalue problem gives the following approximate roots:

$$\begin{aligned} r_1 &= -0.6468621290059189 - 0.6998193343823675i, \\ r_2 &= -0.6468621290059186 + 0.6998193343823683i, \\ r_3 &= -0.2288359770583268, \\ r_4 &= 0.2710936191336444. \end{aligned}$$

All four satisfy

$$|r_k| < 1,$$

so they are inside the prescribed unit disk.

Root plot and residuals

To assess the accuracy of the approximate roots, we compute the residual

$$|f(r_j)|$$

for each computed root.

The residuals are

Root	$ f(r_j) $
r_1	2.34×10^{-14}
r_2	3.56×10^{-14}
r_3	1.47×10^{-13}
r_4	9.98×10^{-14}

All four residuals are very small. The largest residual is approximately

$$9.98 \times 10^{-14}.$$

Therefore, the generalized eigenvalue method produces highly accurate approximations to the roots of f .

The approximate roots are plotted in the complex plane in Figure 1. The unit circle is also shown to verify that all four roots lie inside the prescribed contour.

(c) Root Refinement using Newton's Method

A simple way to improve the accuracy is to follow up the DL/BH method with Newton's method. For each approximate root r_j obtained in part (b), Newton's method is applied using the initial guess

$$z_0 := r_j.$$

Newton's iteration is given by

$$z_{n+1} = z_n - \frac{f(z_n)}{f'(z_n)},$$

where

$$f(z) = e^{2z} \cos(5z) - 5z^2$$

and

$$f'(z) = e^{2z} (2 \cos(5z) - 5 \sin(5z)) - 10z.$$

Using `scipy.optimize.newton`, the approximate roots from part (b) were refined to

$$\begin{aligned} r_1 &\approx -0.6468621290059199 - 0.6998193343823674i, \\ r_2 &\approx -0.6468621290059199 + 0.6998193343823674i, \\ r_3 &\approx -0.2288359770583526, \\ r_4 &\approx 0.2710936191336348. \end{aligned}$$

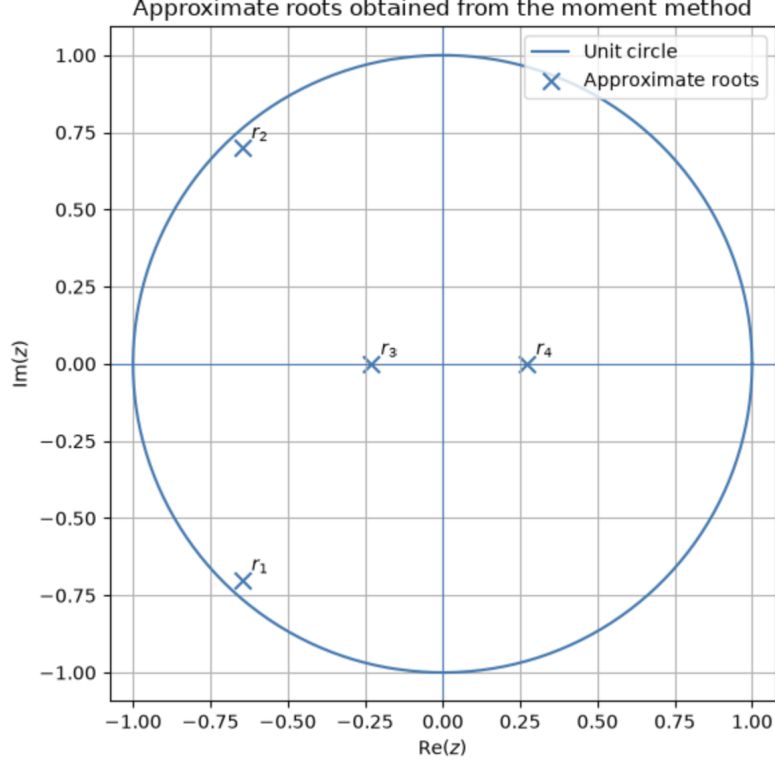


Figure 1: Approximate roots obtained from the generalized eigenvalue problem in Task 1(b). The unit circle is shown for reference.

The residuals after Newton refinement are

$$\begin{aligned} |f(r_1)| &\approx 1.51 \times 10^{-15}, \\ |f(r_2)| &\approx 1.51 \times 10^{-15}, \\ |f(r_3)| &= 0, \\ |f(r_4)| &\approx 5.55 \times 10^{-17}. \end{aligned}$$

The approximation quality improves after applying Newton's method. The roots obtained in part (b) are already good approximations, but Newton's method refines them further.

For example, one of the roots obtained in part (b) was

$$r_3 \approx -0.2288359770583268,$$

whereas after Newton refinement we obtain

$$r_3 \approx -0.2288359770583526.$$

The improvement can also be seen from the residuals

$$|f(r_j)|.$$

The residuals before and after Newton refinement are:

Root	Before Newton	After Newton
r_1	2.34×10^{-14}	1.51×10^{-15}
r_2	3.56×10^{-14}	1.51×10^{-15}
r_3	1.47×10^{-13}	0
r_4	9.98×10^{-14}	5.55×10^{-17}

The residuals become smaller after applying Newton's method. The approximation from 1(b) was already very accurate, but Newton's method improves/refines the approximation, as shown by the smaller residual.

(d) Guessing the number of roots

If the number of roots is not known, the DL/BH method requires a guess N_e . For each value of N_e , $2N_e$ moments were computed and used to construct the Hankel matrices

$$H_0 = (S_{i+j})_{i,j=0}^{N_e-1}, \quad H_1 = (S_{i+j+1})_{i,j=0}^{N_e-1}.$$

The generalized eigenvalue problem

$$H_1 v = \lambda H_0 v$$

was then solved, with the eigenvalues λ giving the approximate roots. The condition number of H_0 was calculated using

$$\kappa(H_0) = \text{numpy.linalg.cond}(H_0).$$

The calculation was repeated for

$$N_e = 1, 2, \dots, 8.$$

The approximate roots obtained for the different values of N_e are shown in Figure 2. The corresponding condition numbers are given in Table 2 and plotted in Figure 3.

Table 2: Condition number of H_0 for different values of N_e .

N_e	$\kappa(H_0)$
1	1.00
2	1.16×10^1
3	9.93
4	1.64×10^2
5	8.03×10^{15}
6	2.10×10^{16}
7	3.29×10^{16}
8	5.28×10^{16}

For smaller values of N_e , fewer approximate eigenvalues are obtained, while for larger values of N_e , additional eigenvalues appear.

The condition number does not by itself provide a criterion for selecting the number of roots. For $N_e = 1, 2, 3$, and 4, the condition numbers remain relatively small compared with those obtained for $N_e \geq 5$. In particular, the condition number is not monotonic in N_e , since $N_e = 3$ gives $\kappa(H_0) \approx 9.93$, which is smaller than the value obtained for $N_e = 2$ and $N_e = 4$. Therefore, a small condition number alone cannot be used to determine the number of roots. Also when $N_e = 2$, the generalized eigenvalue problem produces the approximate roots

$$-0.35106896 \quad \text{and} \quad -2.984939925156.$$

The second value has modulus approximately 2.985, so it lies outside the unit disk even though the condition number is small.

For $N_e \geq 5$, the condition number increases to approximately 10^{15} – 10^{16} , indicating severe ill-conditioning of H_0 . The generalized eigenvalue problem consequently becomes numerically unstable and produces unreliable additional eigenvalues.

Thus, without prior knowledge of the number of roots, the results of this experiment show how the stability of the Hankel matrix and the computed eigenvalues depend on the chosen value of N_e , but they do not by themselves determine the exact number of roots.

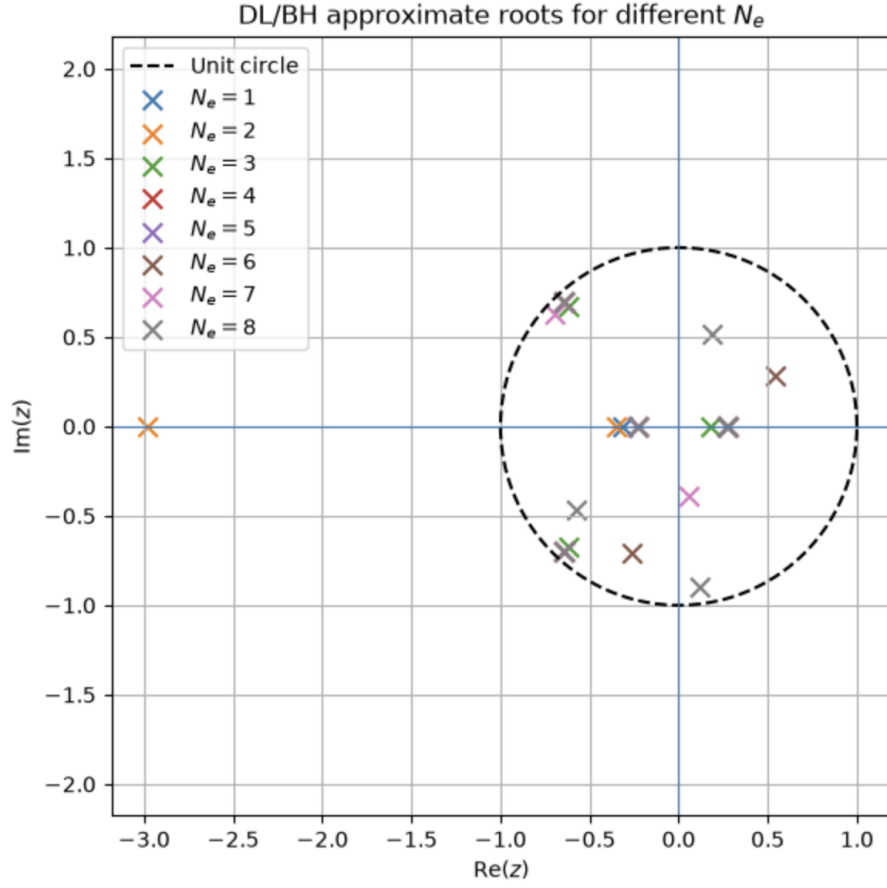


Figure 2: Approximate roots obtained from the DL/BH method for $N_e = 1, \dots, 8$.

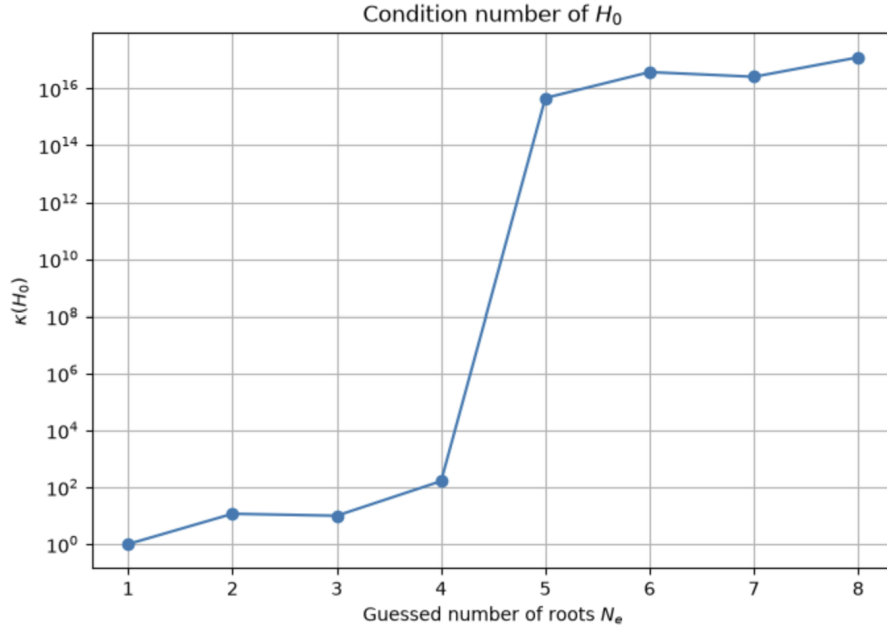


Figure 3: Condition number $\kappa(H_0)$ for different values of N_e .

Task 2: Black-box solution of a nonlinear eigenvalue problem (NLEVP)

We solve an NLEVP on the rectangle

$$K = \{z \in \mathbb{C} : 12500 < \Re z < 112500, \quad 0 < \Im z < 10000\}.$$

The nonlinear eigenvalue problem is

$$f(\lambda) := \det \left(S - \lambda M + i\sqrt{\lambda - \sigma_1} W_1 + i\sqrt{\lambda - \sigma_2} W_2 \right) = 0.$$

We define

$$F(\lambda) = S - \lambda M + i\sqrt{\lambda - \sigma_1} W_1 + i\sqrt{\lambda - \sigma_2} W_2,$$

where S , M , W_1 , and W_2 are large sparse matrices.

Therefore,

$$\boxed{f(\lambda) = \det F(\lambda) = 0.} \tag{1}$$

Instead of solving the determinant problem directly, we consider the scalar function.

$$\boxed{g(z) = v^\top F(z)^{-1} w,} \tag{2}$$

where v and w are fixed vectors.

Then

$$\boxed{\lim_{z \rightarrow \lambda} v^\top F(z)^{-1} w = \infty.}$$

Equivalently,

$$\boxed{\lim_{z \rightarrow \lambda} g(z) = \infty.}$$

The scalar parameters used in the implementation are

$$\sigma_1 = 0, \quad \sigma_2 = 11854.2882308.$$

(a) Relation Between Poles of $g(z)$ and Eigenvalues

Proof that any λ solving (2) must also solve (1)

Given

$$f(z) = \det(F(z))$$

and

$$g(z) = v^\top F(z)^{-1} w.$$

Suppose that $\lambda \in K$ solves equation (2). Then, by definition,

$$\lim_{z \rightarrow \lambda} g(z) = \lim_{z \rightarrow \lambda} v^\top F(z)^{-1} w = \infty.$$

Using the Algebraic Formula for the Inverse of a Matrix in terms of its Adjugate Matrix and Determinant,

$$F(z)^{-1} = \frac{1}{\det(F(z))} \operatorname{adj}(F(z)).$$

Substituting this into the expression for $g(z)$ gives

$$g(z) = \frac{v^\top \operatorname{adj}(F(z)) w}{\det(F(z))}.$$

Since the matrix components S, M, W_1, W_2 are constant and the square-root branches are analytic on K , $F(z)$ is analytic on K .

Since $\lambda \in K$ and K is open, there exists a neighbourhood of λ contained in K . Hence, $F(z)$ is continuous in a neighbourhood of λ , and its entries remain bounded as $z \rightarrow \lambda$.

The entries of $\text{adj}(F(z))$ are determinants of submatrices of $F(z)$. Therefore, they are also continuous and bounded in a neighbourhood of λ .

Since v and w are fixed, finite vectors, the numerator

$$v^T \text{adj}(F(z))w$$

remains bounded as $z \rightarrow \lambda$.

For the fraction to approach infinity as $z \rightarrow \lambda$, the denominator must therefore approach zero:

$$\lim_{z \rightarrow \lambda} \det(F(z)) = 0.$$

Since $\det(F(z))$ is continuous,

$$\det(F(\lambda)) = 0.$$

Since

$$f(z) = \det(F(z)),$$

we have

$$f(\lambda) = \det(F(\lambda)) = 0.$$

Thus, λ must be a solution of equation (1).

Argument for why the reverse may not always be true

Suppose λ satisfies equation (1), meaning

$$\det(F(\lambda)) = 0.$$

This means that $F(\lambda)$ is a singular matrix. However, this does not guarantee that

$$\lim_{z \rightarrow \lambda} g(z) = \infty.$$

From the expression

$$g(z) = \frac{v^T \text{adj}(F(z))w}{\det(F(z))},$$

the singularity can be cancelled if the numerator also approaches zero.

In particular, cancellation can occur if

$$\lim_{z \rightarrow \lambda} v^T \text{adj}(F(z))w = 0.$$

If the numerator has a zero at λ of the same order as, or higher order than, the zero of

$$\det(F(z)),$$

then the pole can be cancelled. In this case,

$$\lim_{z \rightarrow \lambda} g(z)$$

can remain finite instead of becoming infinite.

Therefore, a solution of equation (1) does not necessarily produce a pole of $g(z)$. The occurrence of a pole can depend on the choice of the vectors v and w .

Hence, every solution of (2) is a solution of (1), but the reverse implication does not necessarily hold.

(b) Rational Approximation Using AAA

The provided file `g_eval.csv` contains a finite set of sampling points and the corresponding evaluations of $g(z)$. The first two columns contain the real and imaginary parts of the sampling points z_j , while the last two columns contain the real and imaginary parts of $g(z_j)$.

The complex sampling points and function values were therefore constructed as

$$z_j = \text{Re}(z_j) + i \text{Im}(z_j),$$

and

$$g_j = \text{Re}(g(z_j)) + i \text{Im}(g(z_j)).$$

In Python, these data were loaded and converted into the complex vectors

$$z_{\text{eval}} = (z_1, \dots, z_J)^T$$

and

$$g_{\text{eval}} = (g(z_1), \dots, g(z_J))^T.$$

The AAA algorithm was then applied to these data using `scipy.interpolate.AAA`. This constructs a rational approximation

$$r(z) \approx g(z).$$

The resulting rational function provides a surrogate for the expensive function $g(z)$. Instead of repeatedly evaluating $g(z) = v^T F(z)^{-1} w$, which requires solving a large sparse linear system, the rational surrogate $r(z)$ can be evaluated much more cheaply.

For the given dataset, there are 1111 sampling points. The AAA algorithm selected 41 support points and produced a rational approximation with a final training error of approximately

$$2.57 \times 10^{-12}.$$

Thus, the AAA approximation provides a highly accurate surrogate for the sampled values of $g(z)$ and can subsequently be used to locate candidate poles, which approximate solutions of the nonlinear eigenvalue problem.

(c) Pole Identification and Residual Validation

The poles of the rational approximation were extracted using

$$\{\lambda_j\}_j = r.\text{poles()}.$$

The AAA approximation produced 40 poles in total.

The poles were plotted in the complex plane, together with the region

$$K = \{z \in \mathbb{C} : 12500 < \text{Re}(z) < 112500, \quad 0 < \text{Im}(z) < 10000\}.$$

Figure 4 shows all the poles obtained from the AAA approximation. Figure 5 shows a magnified view of the region K .

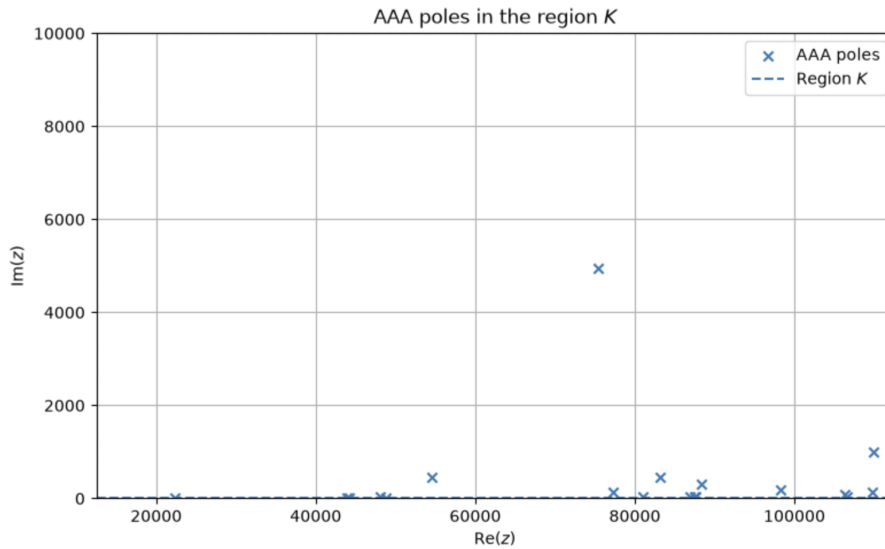


Figure 5: AAA poles in the region K .

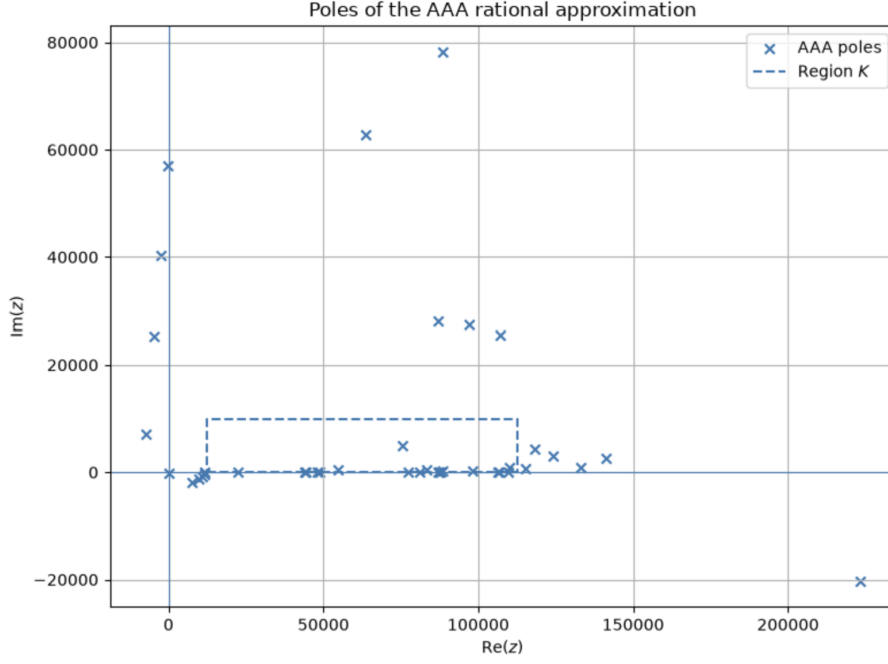


Figure 4: All poles of the AAA rational approximation. The dashed rectangle represents the region K .

From the computed poles, 7 poles have negative imaginary parts. These are

$$\begin{aligned}
 &223248.75 - 20207.27i, \\
 &154.13 - 69.21i, \\
 &7720.62 - 1834.36i, \\
 &9854.38 - 1319.41i, \\
 &10881.68 - 717.44i, \\
 &11459.47 - 305.34i, \\
 &11753.50 - 66.26i.
 \end{aligned}$$

For physical reasons, eigenvalues cannot have negative imaginary parts. Therefore, these poles cannot represent physical eigenvalues.

To further assess the accuracy of the poles, the residual

$$\rho_j = \frac{1}{|g(\lambda_j)|}$$

was computed for every pole λ_j . A small value of ρ_j means that $|g(\lambda_j)|$ is large, which is consistent with λ_j being close to a pole of g .

There are 19 poles inside the region K . Their real parts, imaginary parts, and direct residuals are shown in Table 3.

The residuals for the 19 poles inside K are generally well supported by the residual test and small. They range from approximately

$$9.15 \times 10^{-14} \quad \text{to} \quad 1.50 \times 10^{-8}.$$

Most of the residuals are extremely small, although the pole

$$\lambda = 87627.511 + 32.131i$$

has the largest residual among the 19 poles,

$$\rho = 1.497 \times 10^{-8}.$$

Table 3: AAA poles inside K and direct residuals.

$\text{Re}(\lambda)$	$\text{Im}(\lambda)$	$1/ g(\lambda) $
22345.117	0.645	5.140×10^{-12}
43857.601	20.526	1.052×10^{-10}
44259.419	3.576	1.279×10^{-11}
48142.069	41.892	2.830×10^{-11}
48788.732	6.324	6.953×10^{-13}
54550.139	459.517	7.595×10^{-12}
75402.853	4948.349	9.151×10^{-14}
77240.790	143.901	4.156×10^{-13}
80991.856	32.387	1.483×10^{-13}
83158.783	458.867	4.329×10^{-13}
86832.892	45.657	3.999×10^{-11}
87407.356	35.982	4.794×10^{-11}
87627.511	32.131	1.497×10^{-8}
88394.770	298.729	1.760×10^{-10}
98263.263	186.127	4.765×10^{-11}
106301.431	86.161	2.081×10^{-11}
106625.999	27.036	1.963×10^{-12}
109835.027	133.732	9.147×10^{-14}
109910.146	998.046	2.382×10^{-13}

Therefore, the 19 poles inside K provide good candidate approximations to the poles of g and hence to solutions of the original nonlinear eigenvalue problem in the region K .

There are also some poles outside K with small residuals. For example,

$$\lambda = 115054.387 + 604.479i$$

has residual

$$\rho = 1.094 \times 10^{-7}.$$

Another example is

$$\lambda = 118072.839 + 4295.040i,$$

which has residual

$$\rho = 2.286 \times 10^{-4}.$$

There is also the pole

$$\lambda = 124224.789 + 3037.155i$$

with residual

$$\rho = 4.158.$$

Although some poles outside K can have small residuals, they should be treated with caution because they lie outside the region of interest and outside the region where the approximation is intended to identify the required eigenvalues. Therefore, the residual test alone does not provide sufficient justification for accepting poles outside K as eigenvalues of the problem.

Overall, the AAA approximation produces 40 poles. Of these, 7 have negative imaginary parts and therefore cannot represent physical eigenvalues. The 19 poles lying inside K have generally small residuals and are therefore well supported by the residual test. Poles outside K , even when their residuals are small, should be treated with caution.

(d) AAA vs DL/BH Methods

The DL/BH method is based on contour integration. The function is evaluated at a number of points on a contour, and these evaluations are used to compute contour integrals and moments. These moments are then used to obtain approximations to the eigenvalues.

The AAA-based approach uses a different strategy. A finite set of evaluations of $g(z)$ is first used to construct a rational approximation

$$r(z) \approx g(z).$$

The poles of the rational approximation can then be extracted and used as candidate eigenvalues.

The main difference is that DL/BH works directly with function evaluations on a contour, whereas AAA uses a set of function evaluations to construct a rational surrogate and then works with the surrogate.

Pros and Cons of the DL/BH and AAA Methods

The main advantage of the DL/BH method is its strong theoretical basis, including root counting from the Argument Principle. The contour-integral formulation provides information about the roots inside the chosen contour, and the moments can be used to obtain the roots through a generalized eigenvalue problem. The method therefore provides a direct connection between the contour integrals and the roots.

A disadvantage of the DL/BH method is that it requires function evaluations at the contour points. When evaluating $g(z)$ is expensive, increasing the number of contour points increases the computational cost. The subsequent construction of the moment and Hankel matrices also contributes to the computational cost.

The main advantage of the AAA method is that it constructs a rational surrogate

$$r(z) \approx g(z)$$

from a finite set of sampled function values. The poles of the surrogate can then be extracted without requiring further evaluations of the original function during the pole extraction step. This can be attractive when evaluations of $g(z)$ are expensive, since the available expensive samples can be reused to construct the surrogate.

However, the AAA method also has disadvantages. The poles of the rational approximation are not automatically guaranteed to correspond to true solutions of the nonlinear eigenvalue problem. The approximation can produce spurious poles, so the candidate poles need to be validated using the original function. In this problem, this is done using the residual

$$\frac{1}{|g(\lambda_j)|}.$$

Furthermore, the quality of the rational approximation depends on the sampling points and the number of samples used.

Therefore, when evaluations of $g(z)$ are very expensive, the main advantage of AAA is the ability to construct a surrogate from a finite set of expensive samples and subsequently work with the surrogate. The DL/BH method, on the other hand, directly uses contour evaluations and has a stronger theoretical connection to the roots, but its computational cost can increase with the number of contour points required.

Complexity of the DL/BH and AAA Method

To compare the computational cost of the two methods, the total execution time of the DL/BH method and the AAA-based method was measured. The DL/BH method took approximately

$$0.231914 \text{ s,}$$

while the AAA construction and pole extraction took approximately

$$1.838682 \text{ s.}$$

Thus, for the particular implementations and settings used in this work, the DL/BH method was faster than the AAA computation. However, the AAA timing uses the supplied `g_eval.csv` file, so it does not include the time required to generate the samples by evaluating $g(z)$. The samples were precomputed and provided as part of the coursework because evaluating $g(z)$ can be expensive.

The computational complexity of the DL/BH method is mainly determined by:

- the number of contour points used;
- the cost of each evaluation of $f(z)$ or $g(z)$;
- the cost of the subsequent linear algebra involving the computed moments.

Therefore, increasing the number of contour points increases the number of expensive function evaluations and hence increases the overall computational cost.

The computational complexity of the AAA method is mainly determined by:

- the number of expensive sample evaluations of $g(z)$;
- the number of support points selected by the AAA algorithm;
- the cost of constructing the rational approximation;
- the cost of extracting the poles of the rational approximation.

Once the rational approximation has been constructed, evaluating the surrogate is relatively inexpensive compared with evaluating $g(z)$ directly.

Therefore, the measured timings alone do not imply that one method is generally faster than the other; they only describe the performance of the implementations used in this experiment.

In conclusion, both methods are therefore affected by the cost and number of evaluations of the nonlinear function. The main difference is that the DL/BH method repeatedly uses function evaluations on a contour, while AAA uses a finite set of expensive samples to construct a rational surrogate and then works mainly with this cheaper surrogate.

This makes the AAA method more attractive when evaluations of f or g are particularly expensive, although its candidate poles require additional validation because the rational approximation can contain spurious poles and it does not provide the same direct guarantees as the contour-based approach. The DL/BH method, in contrast, has the advantage of the theoretical information provided by the contour integral approach, but may require a larger number of expensive function evaluations.